From Spaghetti Code to Maintainable Web Architecture

How to structure a web application so code stays modular, maintainable, and easier to change.

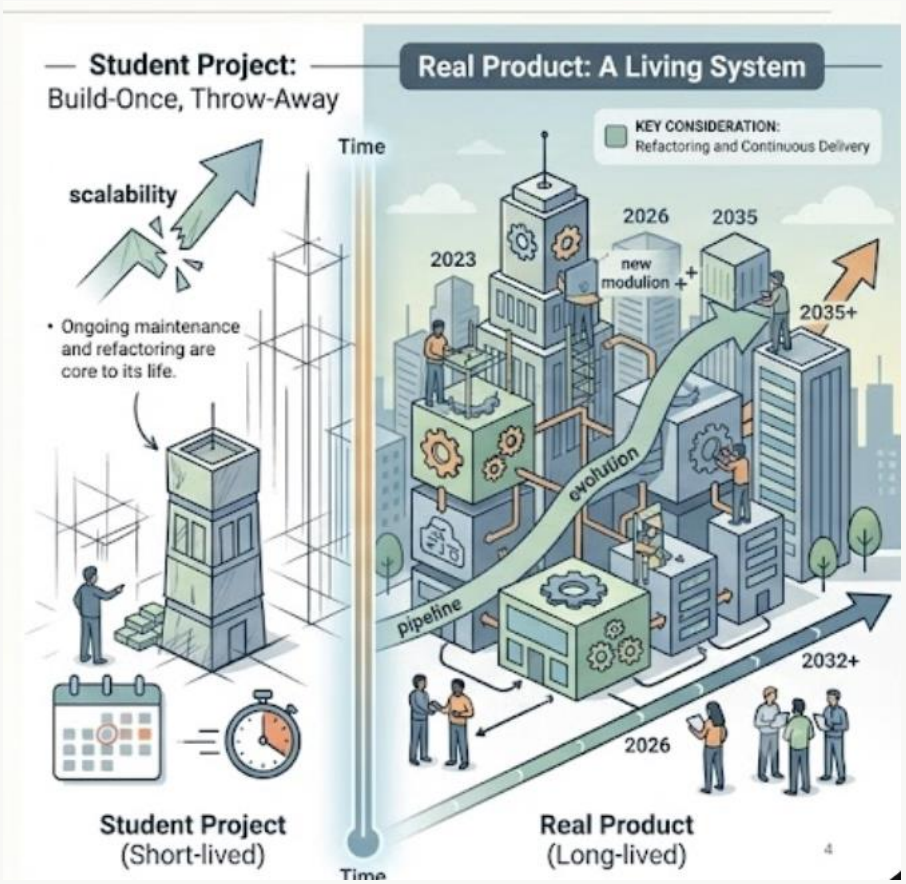
Muhammad Fakhar

Questions for the Audience.

- Have you worked on a project bigger than a solo assignment?
- Have you ever worked on code you did not write yourself?
- Have you ever written code that worked, but later you did not want to touch it?

Student Projects vs. Real Products

- Student projects are usually short-lived.
- Real software may live for years.
- That changes how we think about architecture.



Setup and Learning Goals

- **Know**:
  JavaScript, basic React

- **Have installed:**
  Node.js, npm, Git, code editor

- **Optional accounts:**
  Resend for email demo
  Sentry for telemetry demo

By the end, you can:

- Split messy React code by responsibility
- Wrap and replace an email provider
- Learn about TypeScript Usage and tests for safer changes
- Get started with Sentry debugging

What this seminar is about

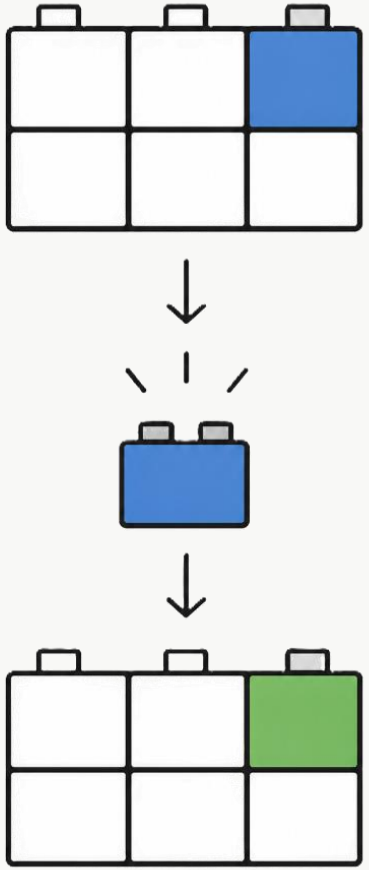
Agenda

- Why maintainability matters
- Simple principles for cleaner web architecture
- Main practical:
  Build an email wrapper and switch providers
- Supporting ideas:
  TypeScript, runtime validation, and testing
- Production debugging:
  What Sentry helps us see
- Bonus practical:
  Refactor a messy React component

Why maintainability matters

- Successful software keeps being used.
- Software that keeps being used keeps changing.

  Software is successful when people use it. But if people use it, they will ask for changes. New requirements arrive.
- Architecture helps us keep that change manageable.
- How to replace external tools safely



Not Just My Opinion

- Lehman's Law:
  Software in use must keep adapting.

- Technical debt:
  Shortcuts can compound over time.

- Martin Fowler:
  Code is read by humans, not only executed by computers.

Clean Code: A Handbook of Agile Software Craftsmanship (Robert C. Martin)

- Code is read much more often than it is written (10:1)
- The only valid measure of code quality is WTFs/minute

Cost of bad architecture

The cost of poor software quality (CPSQ) in the US alone reached $2.41 trillion in 2022, with accumulated technical debt principal estimated at $1.52 trillion.

’Maintenance typically consumes 40 to 80 percent of software costs. It is probably the most important life cycle phase of software.’ (Robert L. Glass) This is why architecture matters—you are optimizing for the 80%, not the 20%.

Knight Capital Group trading glitch (2012)

- The new deployed code repurposed an old order flag.
- They lost $460 million in 45 minutes

Important starting point

There is no single perfect architecture for every web application.

The right architecture depends on:

- How often requirements change
- Time pressure
- Prototype vs. production system
- Security and performance needs

Goal: learn practical principles for better architecture decisions — not one “correct” folder structure.

Core architecture principles

Short theory section before the practical work.

- DRY — do not repeat business rules
- KISS — avoid unnecessary complexity
- Encapsulation — hide internal details
- High cohesion, low coupling — related code together, fewer unnecessary dependencies
- Pure functions — simple, reusable business rules

Example: DRY

DON’T REPEAT YOURSELF: Instead of repeating password validation in multiple places:

```
export function isPasswordStrong(password) {
  return password.length >= 8 && /\d/.test(password);
}
```

Used in:

- Signup form
- Forgot password form
- Profile/Change-password page
- Tests

DRY is not about removing every repeated line. It is about avoiding repeated business knowledge.

Example: KISS

KEEP IT SIMPLE: Good architecture should make the project easier to understand, not harder.

For this seminar, the demo will not use:

- Microservices
- Complex enterprise folder structures
- Too many abstraction layers
- Advanced design patterns everywhere
    - Onion Architecture pattern with Repository-Service Layers.
    - Orchestrated Polyglot Microservices pattern.

We will use a simple structure that makes responsibilities clear.

Example: Encapsulation / Information Hiding

Hide internal details. Expose only what other code needs.

```
// emailService.js
export async function sendWelcomeEmail(email) {
  return Resend.send({
    to: email,
    subject: "Welcome!",
    html: "<h1>Welcome to the app</h1>",
  });
}
```

Other files import from the public interface:

```
import { sendWelcomeEmail } from "./emailService";
```

- Internal files can change later
- Other code does not need to know the internal structure
- The rest of the app is less likely to break

Example: High cohesion and low coupling

High cohesion: related code stays together.

```
signup/
  SignupForm.jsx
  signupApi.js
  validation.js
  index.js
```

Low coupling: unrelated parts do not depend too much on each other.

- The component should not know email provider details.
- The component should not know backend internals
- The component should not depend on unrelated modules

Main idea: code that changes together should live together.

Example: Pure functions for business rules

A pure function takes input, returns output, and does not call APIs, databases, emails, or files.

```
export function calculateDiscountedPrice( price, userType ) {
if (userType === "student") {
return price * 0.8;
}
if (userType === "premium") {
return price * 0.9;
}
return price;
}
```

- Easy to test
- Easy to reuse
- Easy to understand
- Safe to refactor

Practical: Email provider wrapper

Students create a small email wrapper and may try sending a real email using a provider account.

```
src/
  pages/
    PlaceOrderPage.jsx
    ContactSupportPage.jsx
```

```
src/
  pages/
    PlaceOrderPage.jsx
    ContactSupportPage.jsx

  services/
    emailService.js
```

- Today: Resend
- Tomorrow: Mailjet
- Later: SendGrid
- Only provider/wrapper code should change
- The rest of the app stays the same

Total time for practical: 30 minutes

Dependency inversion and wrapper pattern

Problem: business logic should not directly depend on third-party vendors.

BAD: app code knows Resend directly

```
function sendEmailWithResend(to, subject,html){
Resend.send(
  to: email,
  subject: ”…",
  html: "...”)
};
```

BETTER: app code calls our wrapper

```
await myEmailService.sendEmail(email);
```

The application should depend on our own wrapper function, not directly on Resend, we will then swap Resend for MailJet without app breaking.

TypeScript

38% of bugs at Airbnb could have been prevented by TypeScript

Just JavaScript with types

Good for:
- understanding what data should look like
- catching wrong function arguments
- making refactoring safer
- improving autocomplete and documentation
- reducing “what does this object contain?” confusion

- 2nd most loved programming language by devs in stack overflow survey 2020. (written on TS website)

- 85% of respondents use TypeScript more than or equal to JS regularly (state of JS survey 2025)

- 98% of Fortune 500 companies with web applications use TypeScript.

TypeScript and false sense of security

- TypeScript protects you against typos and bad types, not against bad logic.
- TypeScript checks our code while writing, but external data still needs runtime validation.

FRONTEND EXPECTS

```
{
  user: {
    userID: 12353
  }
}
```

BACKEND SENDS

```
{
  user: {
    userID: ”user_12353"
  }
}
```

Validate external data at the boundary: API responses, request bodies, webhooks, env variables, file uploads, third-party APIs.

Testing as architecture protection

Good architecture makes testing easier. Testing makes future changes safer.

Tests protect behavior, not implementation

- Catch Breaking changes when code changes
- Give developers confidence to refactor
- Document how the system is expected to behave
- Protect hidden and legacy functionality

Hidden behaviors — like keyboard shortcuts — can be accidentally broken. Some Engineers may not know the legacy features in the app or some keyboard shortcuts etc, having tests make sure that nothing is broken by mistake.

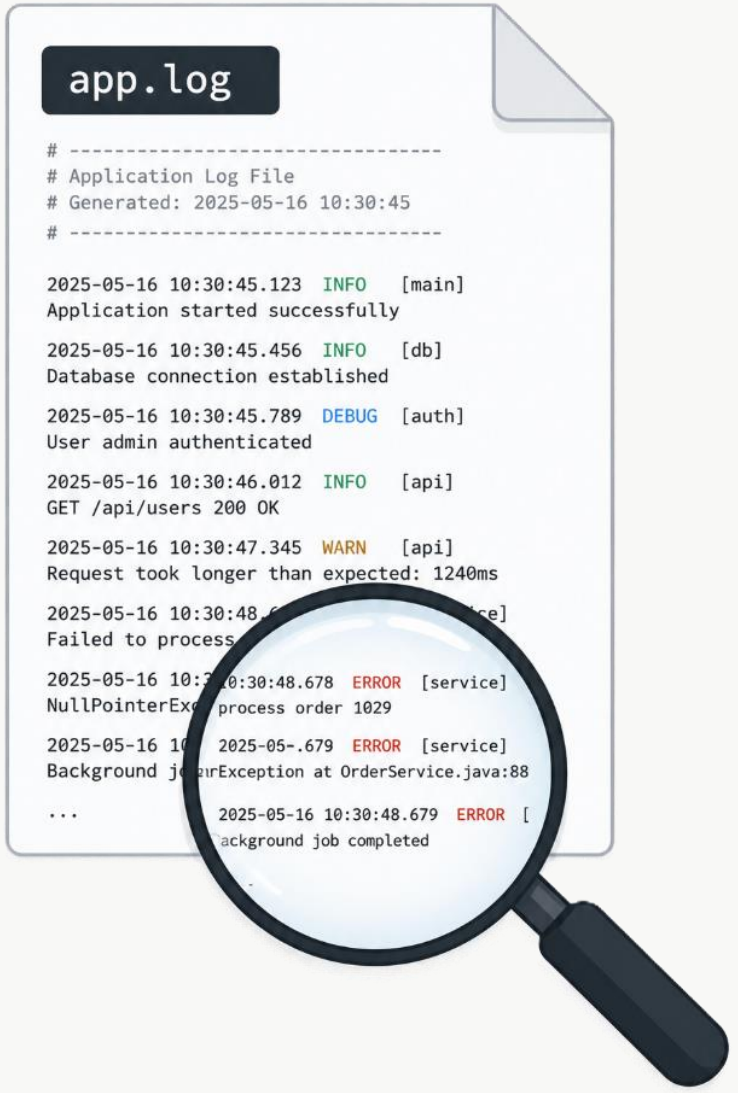
Production telemetry with Sentry

Why Telemetry?

- **74% believe observability is important for monitoring critical business processes** and 65% say it is key to understanding user journeys.
- **72% see positive impact on revenue growth** and 71% note improved operating margins and brand perception. (using OpenTelemetry)

Why Sentry?

- Sentry holds a **78% share** of the error monitoring market, nearly 5x its closest competitor New Relic.
- Anthropic, Disney+, Reddit, Cloudflare, and Riot Games, Siemens, Oracle Corporation, Samsung , Bosch , Allianz



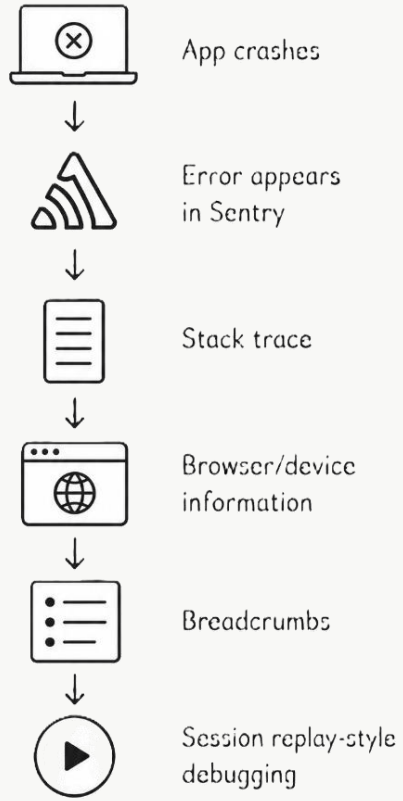
Practical or Demo? Production telemetry with Sentry

Even with clean architecture and tests, bugs can still happen in production.

Telemetry helps answer:

- What error happened?
- Which user action caused it?
- Which browser/device was used?
- Which release introduced the bug?
- Can we reproduce it?

Maintainability continues after deployment.



Practical? Refactor a messy React component

Students get a small messy React repo. The goal is to refactor one feature, not the whole app.

BEFORE

```
Order/ FinalForm3.jsx
// one file does almost everything
```

AFTER

```
src/
  Order/
    OrderForm.jsx
    orderValidation.js
    orderPricing.js
    orderApi.js
    emailMessages.js
```

Refactoring goals

- Move validation rules into orderValidation.js
- Move pricing rules into orderPricing.js
- Move API calls into orderApi.js
- Keep OrderForm.jsx focused on UI and submit flow

Total time for practical: 30 minutes

Final recap: independence is the main theme

- Why maintainability matters
  Successful software keeps changing
- Architecture principles
  DRY, KISS, encapsulation, cohesion, and low coupling help us manage change
- Email wrapper pattern
  the app can switch from Resend to Mailjet without changing the pages
- TypeScript, validation, and testing
  Assumptions become clearer and future changes become safer
- Sentry
  Production bugs become easier to understand and reproduce

Industry Standard Resources

- MDN Web Docs — web platform fundamentals: HTML, CSS, JavaScript, HTTP, Web APIs
- Martin Fowler — software architecture, refactoring, feature flags, design tradeoffs
- OWASP — web application security and secure coding practices
- Official framework docs — React, Next.js, Express, Django, Laravel, Spring Boot, etc.

These sources do not give one universal architecture. They help us make better tradeoffs.

References

Cost of Bad Architecture

[1] Consortium for Information & Software Quality (CISQ). (2022). *The Cost of Poor Software Quality in the US*. https://www.it-cisq.org/the-cost-of-poor-quality-software-in-the-us-a-2022-report/

[2] Glass, R. L. (2002). *Facts and Fallacies of Software Engineering*. Addison-Wesley Professional. https://dokumen.pub/facts-and-fallacies-of-software-engineering-0321117425-5220030051-9780321117427.html

TypeScript Adoption & Bug Prevention

[3] Bunge, B. (Airbnb Engineering). (2019). *Adopting TypeScript at Scale*. JSConf Hawaii. https://youtu.be/P-J9Eg7hJwE?t=712 *(Beweis für die 38% Airbnb-Bug-Statistik ab Minute 11:52)*

[4] Frontscope. (2026). *TypeScript vs JavaScript: Which Should You Learn in 2026?* https://frontscope.dev/blog/typescript-vs-javascript-2026/

[5] State of JavaScript. (2025). *State of JavaScript 2025: Usage & Flavors*. https://2025.stateofjs.com/en-US/usage/#js_ts_balance

Production Telemetry & Sentry Market Share

[6] Cisco/Splunk. (2025). *Splunk Report Shows Observability is a Business Catalyst*. https://investor.cisco.com/news/news-details/2025/Splunk-Report-Shows-Observability-is-a-Business-Catalyst-for-AI-Adoption-Customer-Experience-and-Product-Innovation/default.aspx

[7] Technology Checker. (2026). *Sentry Market Share and Enterprise Adoption*. https://technologychecker.io/technology/sentry

Thank you

Thank you for listening.

Any Questions?